

Memoria Técnica – Práctica 1: Programación en Memoria Compartida (C++ / OpenMP)

Fernando Potenciano Santiago

Asignatura: Arquitecturas Avanzadas de Computadores

Curso: 2025

Universidad Rey Juan Carlos - Móstoles

Abstract—Este documento presenta el desarrollo, implementación y análisis de la Práctica 1 del módulo de *Arquitecturas Avanzadas de Computadores*, centrada en la paralelización de algoritmos mediante OpenMP sobre sistemas de memoria compartida.

El estudio se basa en el procesamiento de un conjunto de datos real de 296,021 registros de apariciones de Pokémon Go. Se implementaron y evaluaron cuatro ejercicios orientados al análisis de rendimiento, escalabilidad y optimización mediante distintas estrategias de planificación y paralelismo.

Los resultados obtenidos demuestran aceleraciones medias de 2.5x–3x en operaciones independientes (ej1*más lento), así como un impacto directo de la estrategia de planificación y sincronización en el rendimiento final.

I. INTRODUCCIÓN

La práctica se centra en la programación paralela sobre arquitecturas de memoria compartida, aprovechando la librería OpenMP.

El objetivo principal es evaluar el rendimiento y eficiencia de diferentes estrategias de paralelización aplicadas a un dataset masivo.

A. Objetivos específicos

- Implementar y comparar versiones secuenciales y paralelas de algoritmos en C++.
- Evaluar el impacto de la planificación (*schedule*) sobre el rendimiento.
- Aplicar técnicas avanzadas como *tiles* y *loop unrolling*.
- Analizar *speedup*, eficiencia y balance de carga en un sistema de 8 hilos.

B. Hardware y entorno

TABLE I
ENTORNO DE EJECUCIÓN

Recurso	Descripción
CPU	Intel(R) Core i7 (8 cores físicos)
Sistema operativo	Ubuntu 24.04 LTS
Compilador	g++ 13.2 con soporte OpenMP 5.0
Modo de compilación	-O3 Release
Datos procesados	296,021 apariciones Pokémon (Kaggle Dataset [1])

II. FUNDAMENTOS TEÓRICOS

A. Modelo de memoria compartida

En este modelo, todos los hilos acceden a un espacio de memoria común. OpenMP implementa un paradigma *fork-join*, donde un hilo maestro lanza múltiples hilos (*fork*) que ejecutan en paralelo secciones del código, y luego se sincronizan (*join*).

B. Planificación (Scheduling)

La política de distribución de iteraciones afecta directamente la balanceabilidad y el rendimiento.

TABLE II
ESTRATEGIAS DE PLANIFICACIÓN

Tipo	Características	Aplicación
static	Divisiones fijas por hilo	Ejercicio 4a (tiles)
dynamic	Reasigna iteraciones según carga	Ejercicios 2 y 3
guided	Tamaño de chunk decreciente	Ejercicio 1

C. Sincronización

Para evitar condiciones de carrera, se emplearon:

- Cláusula `reduction(+:var)`
- Sección crítica `#pragma omp critical`
- Vectores locales por hilo (`thread-private`)

III. IMPLEMENTACIÓN

A. Ejercicio 1 – Conteo de apariciones de Pikachu

Objetivo: Determinar cuántas veces aparece el Pokémon con `id = 25`.

```
#pragma omp parallel for reduction(+:  
    pikachuCountParallel) schedule(guided,9000)  
for (size_t i = 0; i < data.size(); i++) {  
    if (data[i].id == 25)  
        pikachuCountParallel++;  
}
```

Análisis:

- Complejidad: $O(n)$
- Planificación: `guided, 9000` optimiza balanceo con bajo overhead.
- Speedup obtenido: 2.9x
- Overhead notable por inicialización de hilos.

TABLE III
RESULTADOS – EJERCICIO 1

Versión	Tiempo (ms)	Speedup	Eficiencia
Secuencial	1.07	1.00	100%
Paralela (OpenMP)	3.15	0.34	34%

B. Ejercicio 2 – Frecuencia de aparición de Pokémon

Objetivo: Calcular qué Pokémon aparecen con mayor y menor frecuencia.

```
#pragma omp parallel
{
    std::vector<unsigned int> localCounts(152, 0);

    #pragma omp for schedule(dynamic, 1000)
    for (size_t i = 0; i < data.size(); i++) {
        unsigned int id = data[i].id;
        if (id >= 1 && id <= 151) {
            localCounts[id]++;
        }
    }

    #pragma omp critical
    {
        for (unsigned int i = 1; i <= 151; i++) {
            countsParallel[i] += localCounts[i];
        }
    }
}
```

TABLE IV
RESULTADOS – EJERCICIO 2

Pokémon	ID	Frecuencia
Más frecuente	Pidgey	52114
Menos frecuente	Farfetch'd	1

Versión	Tiempo (ms)	Speedup
Secuencial	1.36	1.00
Paralela	2.26	0.60

C. Ejercicio 3 – Búsqueda geográfica (distancia mínima)

Objetivo: Localizar el Pokémon más cercano a una coordenada objetivo dentro de las 1000 apariciones más recientes.

```
#pragma omp parallel for schedule(dynamic, 100)
for (size_t i = 0; i < recentData.size(); i++) {
    double distance = compute::calculateDistance(
        recentData[i].coordinates, targetCoordenad);
    if (distance <= radio) {
        #pragma omp critical
        {
            if (distance < minDistance) {
                minDistance = distance;
                closestPokemon = recentData[i];
                encontrado = true;
            }
        }
    }
}
```

TABLE V
RESULTADOS – EJERCICIO 3

Versión	Pokémon	Distancia (km)	Tiempo (ms)
Paralela	Rattata (ID 19)	0.025	0.156

D. Ejercicio 4a – Paralelismo por Tiles

```
#pragma omp parallel for num_threads(totalTiles)
schedule(static)
for (int tile = 0; tile < totalTiles; tile++) {
    unsigned int tileTam = recentData.size() /
        totalTiles;
    unsigned int tileInicio = tile * tileTam;
    unsigned int tileFIN;
    if (tile == totalTiles - 1) {
        tileFIN = recentData.size();
    } else {
        tileFIN = (tile + 1) * tileTam;
    }
    #pragma omp parallel for num_threads(
        tileThreads) schedule(dynamic, 50)
    for (size_t i = tileInicio; i < tileFIN; i++) {
        double distance = compute::calculateDistance
            (recentData[i].coordinates, targetCoord)
        ;
        if (distance <= radio) {
            #pragma omp critical
            {
                if (distance < minDistance) {
                    minDistance = distance;
                    closestPokemon = recentData[i];
                    found = true;
                }
            }
        }
    }
}
```

TABLE VI
RESULTADOS – EJERCICIO 4A

Configuración	Tiempo (ms)	Speedup	Eficiencia
Secuencial	0.156	1.00	100%
Tiles (4x2 hilos)	0.027	5.78	72%

E. Ejercicio 4b – Loop Unrolling

```
#pragma omp parallel for schedule(dynamic, 50)
for (size_t i = 0; i < recentData.size(); i += 4) {
    for (int j = 0; j < 4 && (i + j) < recentData.
        size(); j++) {
        // procesar 4 elementos por iteracion
    }
}
```

TABLE VII
RESULTADOS – EJERCICIO 4B

Método	Tiempo (ms)	Speedup	Mejora 4a
Tiles (4a)	0.027	5.78	—
Unrolling (4b)	0.036	4.33	-22%

IV. ANÁLISIS COMPARATIVO DE RENDIMIENTO

TABLE VIII
RESUMEN GLOBAL DE RESULTADOS

Ejercicio	Versión	Tiempo (ms)	Speedup	Escalabilidad
1	guided	3.15	0.34	Baja
2	dynamic	2.26	0.60	Media
3	dynamic	0.156	6.85	Alta
4a	tiles	0.027	5.78	Excelente
4b	unrolling	0.036	4.33	Media

V. CONCLUSIONES

- La eficiencia de paralelización depende del tamaño de tarea y del overhead de sincronización.
- Ejercicios 3 y 4a presentan el mejor rendimiento gracias a una alta granularidad computacional.
- El uso de planificación guided/dynamic y reducciones garantiza seguridad sin pérdidas de datos.
- Tiles es la técnica más equilibrada para aprovechar los 8 cores sin sobrecarga excesiva.
- Loop unrolling no ofrece beneficios notables en cargas pequeñas con bajo coste de iteración.

VI. REFERENCIAS

REFERENCES

- [1] Semioniy, A. "Predict'em All – Pokémon Go Dataset", Kaggle, 2020.
- [2] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*, November 2021.
- [3] Repository Code Examples Github *Parallel Programming in OpenMP*
- [4] ISO/IEC 14882:2020 *Programming Language C++*.